

# Topological (World) State-Space Models

Elvin Isufi

Intelligent Systems Department

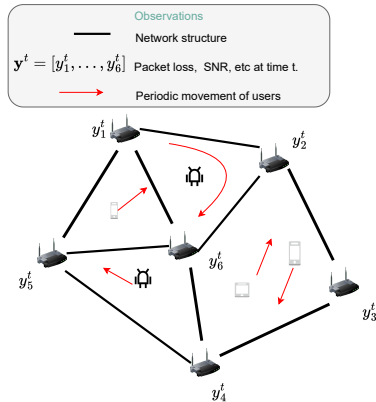
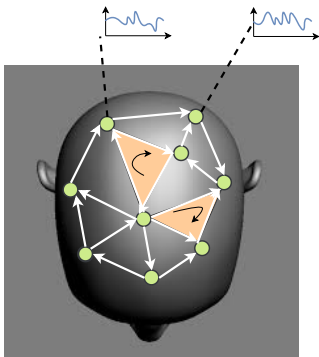
May 15, 2026

# Topological VAR models

- ▶ Topological VAR incorporates the structure into a recursive model
  - ⇒ All happens in the observation space
  - ⇒ Dynamics are often driven in the latent space
- ▶ Today is getting hot world-models under JEPA
  - ⇒ Consider a latent dynamic governing the observation space

# Hidden topological influence

- ▶ Measured time-series often reflect observations of an underlying hidden process
- ▶ We may have missing values in the observations



# Aim

- Model the hidden process using the topological bias
  - ⇒ Use Kalman filter to obtain an accurate representation of the hidden state dynamics

$$\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathcal{L}) + \mathbf{q}_t$$

$$\mathbf{y}_t = g(\mathbf{x}_t, \mathcal{L}) + \mathbf{v}_t$$

- **Objective.** **Detail** the **hidden** state function  $f(\cdot)$  and the **observation** state function  $g(\cdot)$ 
  - ⇒ Facilitate learning of parameters and dynamics
  - ⇒ Handle the high-dimensionality of the problem
  - ⇒ Capture topological properties in  $f(\cdot)$  and  $g(\cdot)$

# Outline: What is this lecture about?

- ▶ How to track latent dynamics over topologies?

⇒ Graph Kalman Networks

⇒ Cellular Complex Kalman Filtering

# Graph Kalman Networks

# Goal of this part

- ▶ We study **time-varying graph signals** observed only partially and noisily.

- ▶ **How can we combine**

graph dynamics      +      state-space inference      +      learnable neural modules  
to predict, interpolate, and track graph time series?

- ▶ **GKNet**: a graph Kalman network based on a graph-aware latent state-space model  
⇒ Useful when training data are scarce, observations are missing, or the graph is uncertain

# Time series over graphs

- ▶ Let  $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{A})$  be a graph with  $N$  nodes and  $M$  edges.

- ⇒  $\mathbf{A} \in \mathbb{R}^{N \times N}$ : adjacency matrix.

- ⇒  $\mathbf{L} \in \mathbb{R}^{N \times N}$ : graph Laplacian.

- ⇒  $\mathbf{B} \in \mathbb{R}^{N \times M}$ : node-edge incidence matrix.

- ▶ A graph time series is a sequence

$$\mathbf{x}_t \in \mathbb{R}^N, \quad t = 1, \dots, T,$$

where  $\mathbf{x}_t[n]$  is the signal value at node  $n$  and time  $t$ .

- ▶ **Tasks**

- ⇒ Forecasting: predict future graph signals.

- ⇒ Interpolation: recover missing node values.

- ⇒ Tracking: infer a hidden graph state from noisy observations.



# State and observation models on graphs

- ▶ A direct graph-temporal model writes

$$\mathbf{x}_t = f(\mathbf{A}, \mathbf{x}_{t-1}) + \mathbf{w}_t.$$

⇒  $f(\cdot)$  describes how the graph signal evolves over the graph

⇒ Restrictive when the measured process is only an indirect view of the underlying dynamics.

- ▶ Latent graph state:

$$\mathbf{x}_t = f(\mathbf{A}, \mathbf{x}_{t-1}) + \mathbf{w}_t,$$

$$\mathbf{y}_t = g(\mathbf{A}, \mathbf{x}_t) + \mathbf{v}_t.$$

⇒  $\mathbf{x}_t \in \mathbb{R}^N$  is the hidden state.

⇒  $\mathbf{y}_t \in \mathbb{R}^{N_o}$  is the partially observed measurement, with  $N_o \leq N$ .

- ▶ Key idea

⇒ Observations may be complicated, but the hidden state is a simple graph-aware dynamics.

# Why not only a spatiotemporal GNN?

- ▶ Standard spatiotemporal GNNs often learn a direct map

$$(\mathbf{y}_{t-T+1}, \dots, \mathbf{y}_t, \mathcal{G}) \mapsto \mathbf{y}_{t+h}.$$

- ▶ This works well with enough data, but it may struggle when:
  - ⇒ training data are limited;
  - ⇒ many node values are missing;
  - ⇒ the graph is imperfect or noisy;
  - ⇒ temporal dynamics should remain interpretable;
  - ⇒ the graph should model the process dynamics, not only the convolution support.

- ▶ **Key idea**

- ⇒ The graph defines a latent dynamical prior; neural modules increase expressivity.

# State model on graphs: heat diffusion

- ▶ The starting point is graph heat diffusion:

$$d\mathbf{x}_t = -c\mathbf{L}\mathbf{x}_t dt.$$

- ⇒  $\mathbf{L}$  controls graph signal smoothing
- ⇒  $c > 0$  is the diffusivity coefficient.
- ⇒ The dynamics are **linear**, **time-invariant**, and **Markovian**

- ▶ **Why heat diffusion?**

- ⇒ It gives a simple graph-aware latent evolution.
- ⇒ It is compatible with recursive state-space inference.
- ⇒ It provides a strong inductive bias when data are scarce.

- ▶ But pure heat diffusion is too rigid:

- ⇒ real graph time series do not exactly diffuse;
- ⇒ graph weights may be uncertain;
- ⇒ the latent model needs controlled degrees of freedom.

# State model on graphs: stochastic diffusion

- ▶ The stochastic graph dynamics is

$$d\mathbf{x}_t = -c\mathbf{L}\mathbf{x}_t dt + \mathbf{S} d\boldsymbol{\beta}_t,$$

where  $\boldsymbol{\beta}_t$  is a Brownian motion and  $\mathbf{S}$  distributes uncertainty over the graph.

- ▶ A general matrix  $\mathbf{S}$  is expressive but problematic:
  - ⇒ it has many degrees of freedom;
  - ⇒ it can overfit when data are scarce;
  - ⇒ it loses graph interpretability;
  - ⇒ it scales poorly with the graph size;
  - ⇒ it is not naturally transferable across graphs.

- ▶ **Design principle**

- ⇒ Do not use arbitrary uncertainty. Tie the uncertainty to graph structure.

## State model on graphs: edge uncertainty

- Replace the dense dispersion matrix by an **edge-structured** uncertainty model:

$$d\mathbf{x}_t = -c\mathbf{L}\mathbf{x}_t dt + \mathbf{B} \text{diag}(\boldsymbol{\alpha}) d\boldsymbol{\beta}_t.$$

$\Rightarrow \mathbf{B} \in \mathbb{R}^{N \times M}$  maps edge quantities to node quantities.

$\Rightarrow \boldsymbol{\alpha} \in \mathbb{R}^M$  contains one uncertainty coefficient per edge.

- **Discrete-time state model**

$$\mathbf{x}_{t+1} = (\mathbf{I} - c\mathbf{L})\mathbf{x}_t + \mathbf{B} \text{diag}(\boldsymbol{\alpha})\mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

- **Large  $\alpha_m$** : the latent process can deviate more strongly from heat diffusion around edge  $m$ .

# Why edge uncertainty?

| Generic uncertainty $\mathbf{S}$  | Graph-induced uncertainty $\mathbf{B} \text{diag}(\alpha)$ |
|-----------------------------------|------------------------------------------------------------|
| Dense matrix with many parameters | One uncertainty parameter per edge                         |
| Weakly interpretable              | Uncertainty localized on graph edges                       |
| High overfitting risk             | Graph-structured inductive bias                            |
| Poor scalability                  | Uses sparse graph incidence structure                      |
| Harder to transfer                | Tied to graph topology                                     |

## ► Interpretation

- ⇒ The graph Laplacian defines the nominal diffusion.
- ⇒ The incidence matrix defines how edge-level uncertainty perturbs the node dynamics.

# Observation model on graphs

- The observation is a sampled graph-filtered version of the latent state:

$$\mathbf{y}_t = \Phi \left( \sum_{k=0}^K h_k \mathbf{L}^k \mathbf{x}_t \right) + \mathbf{v}_t.$$

$\Rightarrow \Phi \in \{0, 1\}^{N_o \times N}$  is the sampling / masking matrix.

$\Rightarrow \tilde{\mathbf{H}}(\mathbf{L}) = \sum_{k=0}^K h_k \mathbf{L}^k$  is a graph filter.

$\Rightarrow \mathbf{v}_t \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$  is observation noise.

- Compact notation

$$\mathbf{y}_t = \mathbf{H} \mathbf{x}_t + \mathbf{v}_t, \quad \mathbf{H} := \Phi \tilde{\mathbf{H}}(\mathbf{L}).$$

- The graph filter allows the observed value at a node to depend on a multi-hop neighborhood of the latent state

# Problem formulation

- ▶ Graph state-space model

$$\begin{aligned}\mathbf{x}_t &= (\mathbf{I} - c\mathbf{L})\mathbf{x}_{t-1} + \mathbf{B} \operatorname{diag}(\boldsymbol{\alpha})\mathbf{w}_t, \\ \mathbf{y}_t &= \mathbf{H}\mathbf{x}_t + \mathbf{v}_t = \Phi\tilde{\mathbf{H}}(\mathbf{L})\mathbf{x}_t + \mathbf{v}_t.\end{aligned}$$

- ▶ with

$$\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \sigma_0^2 \mathbf{I}), \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad \mathbf{v}_t \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}).$$

- ▶ Unknowns to learn:

$$\mathbf{h} = [h_0, \dots, h_K]^\top, \quad \boldsymbol{\alpha} \in \mathbb{R}^M, \quad c, \quad \sigma^2, \quad \mathbf{x}_{0:T}.$$



# Learning graph state-space models

- Define the transition matrix and process covariance

$$\tilde{\mathbf{L}} = \mathbf{I} - c\mathbf{L}, \quad \mathbf{Q} = \mathbf{B} \operatorname{diag}(\alpha^2) \mathbf{B}^\top.$$

- Then the state and observation densities are

$$p(\mathbf{x}_t | \mathbf{x}_{t-1}, \alpha) = \mathcal{N}(\mathbf{x}_t | \tilde{\mathbf{L}}\mathbf{x}_{t-1}, \mathbf{Q}),$$

$$p(\mathbf{y}_t | \mathbf{x}_t, \mathbf{h}, \sigma^2) = \mathcal{N}(\mathbf{y}_t | \mathbf{H}\mathbf{x}_t, \sigma^2 \mathbf{I}).$$

- Likelihood factorization

$$p(\mathbf{y}_{1:T}, \mathbf{x}_{0:T}) = p(\mathbf{x}_0) \prod_{t=1}^T p(\mathbf{y}_t | \mathbf{x}_t) p(\mathbf{x}_t | \mathbf{x}_{t-1}).$$

# Negative log-likelihood

- Up to constants, the negative log-likelihood contains two dominant terms:

$$\mathcal{L} = \underbrace{\frac{1}{2\sigma^2} \sum_{t=1}^T \|\mathbf{y}_t - \mathbf{H}\mathbf{x}_t\|_2^2}_{\text{observation fit}} + \underbrace{\frac{1}{2} \sum_{t=1}^T \|\mathbf{x}_t - \tilde{\mathbf{L}}\mathbf{x}_{t-1}\|_{\mathbf{Q}^{-1}}^2}_{\text{diffusion consistency}} + \dots$$

where

$$\|\mathbf{z}\|_{\mathbf{Q}^{-1}}^2 = \mathbf{z}^\top \mathbf{Q}^{-1} \mathbf{z}.$$

- **Interpretation**

- ⇒ First term: explain the observed measurements.
- ⇒ Second term: keep the latent trajectory close to graph diffusion, except where edge uncertainty allows deviations.

# Why expectation-maximization?

- ▶ The learning problem is circular:

- ⇒ to estimate the hidden states  $\mathbf{x}_{0:T}$ , we need the parameters;
  - ⇒ to estimate the parameters, we need the hidden states.

- ▶ **EM resolves this circularity**

- ⇒ **E-step**: infer the latent state statistics using the current parameters.
    - ⇒ **M-step**: update the parameters using the inferred state statistics.

- ▶ For linear-Gaussian state-space models, the E-step is performed by Kalman filtering and smoothing.

- ▶ **Intuition**

- ⇒ Kalman filtering alternates between predicting the latent state with the model and correcting it using the new observation.

## E-step: Kalman prediction and correction

- Prediction step

$$\mathbf{x}_t^- = \tilde{\mathbf{L}}\mathbf{x}_{t-1}^+, \quad \mathbf{P}_t^- = \tilde{\mathbf{L}}\mathbf{P}_{t-1}^+\tilde{\mathbf{L}}^\top + \mathbf{Q}.$$

- Correction step

$$\mathbf{K}_t = \mathbf{P}_t^- \mathbf{H}^\top \left( \mathbf{H} \mathbf{P}_t^- \mathbf{H}^\top + \sigma^2 \mathbf{I} \right)^{-1},$$

$$\mathbf{x}_t^+ = \mathbf{x}_t^- + \mathbf{K}_t \left( \mathbf{y}_t - \mathbf{H} \mathbf{x}_t^- \right), \quad \mathbf{P}_t^+ = (\mathbf{I} - \mathbf{K}_t \mathbf{H}) \mathbf{P}_t^-.$$

- **$\mathbf{K}_t$** : Kalman gain decides how much to trust the observation residual relative to the model prediction.

## M-step: parameter update

- ▶ After the E-step, we have smoothed latent statistics:  $\hat{\mathbf{x}}_t, \hat{\mathbf{P}}_t, \hat{\mathbf{V}}_t$
- ▶ The M-step minimizes the expected negative log-likelihood:  $\mathcal{Q} = \mathbb{E}[\mathcal{L} \mid \mathbf{y}_{1:T}]$

$$\begin{aligned}\mathcal{Q} = & \frac{NT}{2} \log \sigma^2 + \frac{T}{2} \log |\mathbf{Q}| \\ & + \frac{1}{2\sigma^2} \text{tr} \left( \sum_{t=1}^T \mathbf{y}_t \mathbf{y}_t^\top - \mathbf{H} \hat{\mathbf{x}}_t \mathbf{y}_t^\top - \mathbf{y}_t \hat{\mathbf{x}}_t^\top \mathbf{H}^\top + \mathbf{H} \hat{\mathbf{P}}_t \mathbf{H}^\top \right) \\ & + \frac{1}{2} \text{tr} \left( \mathbf{Q}^{-1} \sum_{t=1}^T \hat{\mathbf{P}}_t - \tilde{\mathbf{L}} \hat{\mathbf{V}}_t - \hat{\mathbf{V}}_t \tilde{\mathbf{L}}^\top + \tilde{\mathbf{L}} \hat{\mathbf{P}}_{t-1} \tilde{\mathbf{L}}^\top \right).\end{aligned}$$

# EM algorithm: summary

- Initialize parameters

$$\mathbf{h}^{(0)}, \quad \boldsymbol{\alpha}^{(0)}, \quad c^{(0)}, \quad \sigma^{(0)}.$$

- Repeat until convergence

⇒ **E-step**: run Kalman filtering and smoothing using current parameters.

$$\hat{\mathbf{x}}_t, \quad \hat{\mathbf{P}}_t, \quad \hat{\mathbf{V}}_t$$

⇒ **M-step**: update the parameters using gradients of  $\mathcal{Q}$ .

$$\mathbf{h}^{(i+1)} = \mathbf{h}^{(i)} - \gamma_1 \nabla_{\mathbf{h}} \mathcal{Q},$$

$$\boldsymbol{\alpha}^{(i+1)} = \boldsymbol{\alpha}^{(i)} - \gamma_2 \nabla_{\boldsymbol{\alpha}} \mathcal{Q},$$

$$c^{(i+1)} = c^{(i)} - \gamma_3 \nabla_c \mathcal{Q},$$

$$\sigma^{(i+1)} = \sigma^{(i)} - \gamma_4 \nabla_{\sigma} \mathcal{Q}.$$

# From EM to GKNet

- ▶ The EM-based method is principled but has limitations:

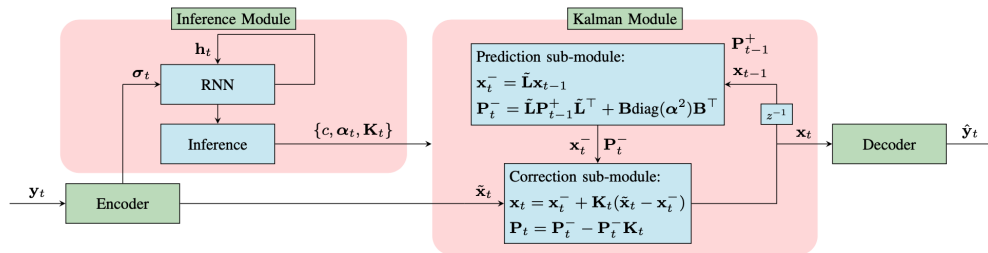
- ⇒ linear state and observation models;
- ⇒ Gaussian assumptions;
- ⇒ matrix inversions in the Kalman recursion;
- ⇒ worst-case complexity of order  $\mathcal{O}(TN^3)$ .

- ▶ **GKNet**

- ⇒ Keeps the graph state-space inductive bias.
- ⇒ Replaces parts of the inference procedure by learnable neural modules.
- ⇒ Uses recurrent inference to learn time-varying parameters.

- ▶ The result is a **model-based graph neural network**:

graph diffusion prior+Kalman prediction–correction+GCNN encoder/decoder+RNN parameter inference.



- The architecture contains four modules:
  - ⇒ **Encoder**: maps observations to the latent state domain.
  - ⇒ **Kalman module**: performs prediction and correction.
  - ⇒ **Inference module**: estimates Kalman parameters recurrently.
  - ⇒ **Decoder**: maps the corrected latent state to the target domain.



# Encoder and decoder modules

- ▶ The linear observation model is replaced by nonlinear GCNN modules.
- ▶ A GCNN layer is

$$\tilde{\mathbf{x}}_t^{(\ell+1)} = \text{ReLU} \left( \sum_{k=0}^K h_{k\ell} \mathbf{L}^k \tilde{\mathbf{x}}_t^{(\ell)} \right).$$

## ▶ Encoder

$$[\tilde{\mathbf{x}}_t, \boldsymbol{\sigma}_t] = \boldsymbol{\Phi}_{\text{enc}}(\mathbf{y}_t, \mathbf{L}; \mathbf{H}_{\text{enc}}).$$

⇒ It returns a rough latent estimate  $\tilde{\mathbf{x}}_t$  and observation uncertainty  $\boldsymbol{\sigma}_t$ .

## ▶ Decoder

$$\hat{\mathbf{y}}_t = \boldsymbol{\Phi}_{\text{dec}}(\mathbf{x}_t, \mathbf{L}; \mathbf{H}_{\text{dec}}).$$

⇒ It maps the corrected latent state back to the prediction or interpolation target.

# Kalman module

► Prediction sub-module

$$\mathbf{x}_t^- = \tilde{\mathbf{L}}\mathbf{x}_{t-1}^+, \quad \mathbf{P}_t^- = \tilde{\mathbf{L}}\mathbf{P}_{t-1}^+\tilde{\mathbf{L}}^\top + \mathbf{B} \operatorname{diag}(\alpha^2)\mathbf{B}^\top.$$

► Correction sub-module

$$\begin{aligned}\mathbf{x}_t^+ &= \mathbf{x}_t^- + \hat{\mathbf{K}}_t(\mathbf{L}) \left( \tilde{\mathbf{x}}_t - \mathbf{x}_t^- \right), \\ \mathbf{P}_t^+ &= (\mathbf{I} - \hat{\mathbf{K}}_t(\mathbf{L}))\mathbf{P}_t^-.\end{aligned}$$

► Main approximation: dense Kalman gain is replaced by a graph filter:

$$\hat{\mathbf{K}}_t(\mathbf{L}) = \sum_{k=0}^K h_{kt}^{(0)} \mathbf{L}^k.$$

# Inference module

- ▶ The inference module is an RNN that produces the parameters of the Kalman module.

$$\mathbf{h}_t = \left[ c_t, \mathbf{h}_t^{(0)\top}, \mathbf{h}_t^{(1)\top} \right]^\top.$$

- ▶ These parameters define:
  - $\Rightarrow c_t$ : graph diffusivity;
  - $\Rightarrow \hat{\mathbf{K}}_t(\mathbf{L})$ : graph-filtered correction gain;
  - $\Rightarrow \boldsymbol{\alpha}_t$ : time-varying edge uncertainty.

- ▶ Edge uncertainty as an edge graph filter

$$\boldsymbol{\alpha}_t = \sum_{k=0}^K h_{kt}^{(1)} (\mathbf{B}^\top \mathbf{B})^k \mathbf{w}_t = \mathbf{H}_e(\mathbf{L}_1) \mathbf{w}_t, \quad \mathbf{L}_1 = \mathbf{B}^\top \mathbf{B}.$$

- ▶ The RNN learns filter coefficients, not a dense state-transition or dense Kalman-gain matrix.

# Training objective

- ▶ The GKNet loss mirrors the likelihood structure:

$$\mathcal{L}_t = \underbrace{\|\hat{\mathbf{y}}_t - \tilde{\mathbf{y}}_t\|_2^2}_{\text{prediction / reconstruction}} + \lambda \underbrace{\|\mathbf{x}_t - \tilde{\mathbf{L}}\mathbf{x}_{t-1}\|_{\mathbf{Q}^{-1}}^2}_{\text{diffusion consistency}} .$$

- ▶ with  $\mathbf{Q} = \mathbf{B} \text{diag}(\boldsymbol{\alpha}^2) \mathbf{B}^\top$

- ▶ **Interpretation**

- $\Rightarrow$  GKNet is trained to fit the data while keeping the latent trajectory close to graph diffusion.
- $\Rightarrow$  The edge uncertainty determines where deviations from diffusion are allowed.

## GKNet: summary

| Component       | Role                                                           |
|-----------------|----------------------------------------------------------------|
| Encoder GCNN    | Maps partial observations to the latent graph-state domain.    |
| Prediction step | Evolves the latent state using graph diffusion.                |
| Correction step | Updates the prediction using a graph-filtered residual.        |
| Inference RNN   | Produces diffusivity, correction filter, and edge uncertainty. |
| Decoder GCNN    | Maps corrected latent states to the target output.             |
| Loss            | Combines data fit and graph-diffusion consistency.             |

- GKNet is a recurrent GNN whose recurrence is organized around Kalman prediction and correction.

# Numerical results: forecasting setup

- ▶ **Task:** multivariate traffic forecasting.
  - ⇒ Given the last  $T$  graph-signal samples, predict the traffic signal at horizon  $h$ .
- ▶ **Datasets**
  - ⇒ METR-LA: traffic data over 207 sensors in Los Angeles.
  - ⇒ PEMS-BAY: traffic data over 325 sensors in the Bay Area.
- ▶ **Graph construction**
  - ⇒ The graph is built from road-network distances using a Gaussian threshold kernel.
- ▶ **Evaluation**
  - ⇒ Normalized root mean square error (nRMSE).
  - ⇒ Lower values are better.
- ▶ **Message**
  - ⇒ In high-data regimes, GKNet is competitive with strong spatiotemporal GNNs.
  - ⇒ In low-data regimes, the model-based inductive bias becomes much more important.

## Numerical results: forecasting

- Example high-data result on METR-LA for prediction horizon  $h = 6$ .

| Method        | nRMSE         |
|---------------|---------------|
| Graph WaveNet | 0.2645        |
| STGCN         | 0.2820        |
| GGRNN         | 0.2489        |
| DCRNN         | 0.2580        |
| GTCNN         | 0.2486        |
| GKNet         | <b>0.2400</b> |

- **Take-home message**

⇒ GKNet can match or improve strong spatiotemporal GNNs while using a structured state-space inductive bias.

# Numerical results: interpolation setup

- **Task:** multivariate time-series interpolation.

⇒ We observe only part of a graph time-series window:

$$\mathbf{Y}_{\text{obs}} = \mathbf{M} \odot \mathbf{Y}.$$

⇒ The goal is to reconstruct the missing node-time values.

- **Datasets**

⇒ Molene: hourly weather data over 32 stations in France.

⇒ NOAA: hourly weather data over 109 stations in the US.

- **Evaluation**

⇒ nRMSE on missing entries.

⇒ Experiments vary the observed node ratio.

- **Why GKNet helps**

⇒ The encoder maps partial observations into a latent graph state.

⇒ The Kalman module regularizes the latent trajectory through uncertain graph diffusion.



## Numerical results: interpolation

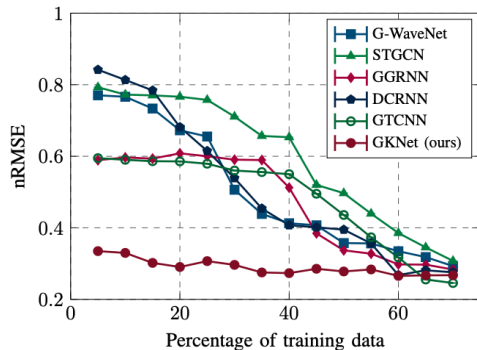
- Example high-data interpolation result on Molene.

| Method        | 90% obs.      | 80% obs.      | 70% obs.      |
|---------------|---------------|---------------|---------------|
| Graph WaveNet | 0.1812        | 0.2164        | 0.2630        |
| STGCN         | 0.1925        | 0.1994        | 0.2380        |
| GGRNN         | 0.1953        | 0.2372        | 0.2449        |
| DCRNN         | 0.2143        | 0.2537        | 0.2649        |
| GTCNN         | 0.1843        | 0.1938        | 0.2411        |
| GKNet         | <b>0.1524</b> | <b>0.1726</b> | <b>0.1758</b> |

- Take-home message

⇒ On this interpolation task, GKNet benefits from explicitly modeling latent dynamics and graph-structured uncertainty.

## Numerical results: low-data interpolation



- Weather interpolation on NOAA under low-data and missing-data regimes.
  - ⇒ GKNet remains stable when the training-data ratio decreases.
  - ⇒ GKNet also remains robust when the observation ratio decreases.

## Ablation: uncertainty and regularization

- The paper studies variants that remove parts of the model.

| Model    | High data   |             |             | Low data    |             |             |
|----------|-------------|-------------|-------------|-------------|-------------|-------------|
|          | 90%         | 80%         | 70%         | 10%         | 20%         | 30%         |
| No-Q     | 0.22        | 0.23        | 0.23        | 0.33        | 0.33        | 0.33        |
| Fixed-Q  | 0.23        | 0.23        | 0.23        | 0.31        | 0.32        | 0.31        |
| No-B     | 0.18        | 0.19        | 0.19        | 0.24        | 0.27        | 0.28        |
| No-Reg-Q | 0.19        | 0.20        | 0.22        | 0.27        | 0.29        | 0.31        |
| GKNet    | <b>0.14</b> | <b>0.17</b> | <b>0.17</b> | <b>0.15</b> | <b>0.17</b> | <b>0.18</b> |

- **Take-home:** Removing edge uncertainty, removing graph structure from uncertainty, or removing the diffusion regularizer hurts performance.

# Cellular Complex Kalman Networks

# Introduction: From Graphs to Cell Complexes

- ▶ **State Estimation Goal:** Infer hidden variables and observations from noisy data with topology
  - ⇒ **The Graph Limitation:** Traditional graphs only process vertex signals and pairwise interactions.
  - ⇒ **Real-World Mismatch:** Physical systems (e.g., water networks) generate tightly coupled signals across multiple dimensions (nodes, edges, faces).
- ▶ **Current Topological Solution and Limitations:**
  - ⇒ **Cell complexes** generalize graphs, using boundary operators to process multi-level signals.
  - ⇒ Current topological time-series models implicitly assume the observations are fully available.
  - ⇒ Topological dynamics are driven by unmeasured, latent variables.
  - ⇒ We need a model that handles incomplete streaming data and dynamically infers these hidden states.

# Introduction: Framework

- ▶ **A Topological State Space Model** designed for streaming, incomplete data.
  - ⇒ **Integrated State Evolution:** Models latent dynamics via a linear SPDE, natively embedding Hodge and Dirac operators for cross-level drift and diffusion.
  - ⇒ **Nonlinear Observations:** Efficiently approximates nonlinear transformations of hidden states using Random Fourier Features (RFF).
  - ⇒ **Online Inference:** Employs a tailored EM and Extended Kalman Filter (EKF) framework for real-time tracking and parameter learning.
  - ⇒ **Topology Discovery:** Introduces a heuristic mechanism to infer unknown higher-order structures directly from data.

# Topology-aware state model

- We consider a topological stochastic differential equation (t-SDE)

$$d\mathbf{x}_t = -c\mathcal{L}\mathbf{x}_t dt + \mathbf{S}d\boldsymbol{\beta}_t$$

⇒ with  $\mathcal{L}$  collecting all Hodge Laplacians

$$\mathcal{L} = \begin{pmatrix} \mathbf{L}_0, & 0, & 0 \\ 0, & \mathbf{L}_1, & 0 \\ 0, & 0, & \mathbf{L}_2 \end{pmatrix}$$

⇒ Simplicial complex signal  $\mathbf{x}_t = [\mathbf{v}_t^\top, \mathbf{f}_t^\top, \boldsymbol{\tau}_t^\top]^\top$

⇒  $\boldsymbol{\beta}_t \in \mathbb{R}^N$  is a standard **Brownian motion**

⇒  $\mathbf{S}$  is an **uncertainty** dispersion **matrix**

⇒ There is **no** structure **information** in  $\mathbf{S}$

# Topology-aware state model: Hierarchical uncertainty

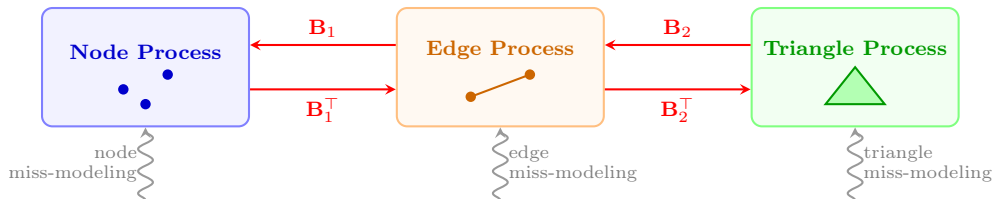
$$d\mathbf{x}_t = -c\mathcal{L}\mathbf{x}_t dt + \underbrace{\mathbf{D}\text{diag}(\boldsymbol{\alpha})}_{\mathbf{S}} d\boldsymbol{\beta}_t$$

- We consider the uncertainty  $\mathbf{S}$  is induced by the **hierarchical** topological connectivity via the Dirac operator  $\mathbf{D}$ :

⇒ **Node** process – due to **edge** miss-modeling ( $\mathbf{B}_1$ )

⇒ **Edge** process – due to **node + triangle** miss-modeling ( $\mathbf{B}_1^\top + \mathbf{B}_2$ )

⇒ **Triangle** process – due to **edge** miss-modeling ( $\mathbf{B}_2^\top$ )





# Topology-aware state model: Dirac Operator

- **Solution:** Uncertainty affects simplicial signals via the boundary maps.

⇒ We replace the dense matrix **S** with  **$\mathbf{D}_{\mathcal{X}} \text{diag}(\boldsymbol{\alpha})$** :

$$d\mathbf{x}_t = -c\mathcal{L}\mathbf{x}_t dt + \mathbf{D}_{\mathcal{X}} \text{diag}(\boldsymbol{\alpha}) d\boldsymbol{\beta}_t$$



- ⇒  $\boldsymbol{\alpha} \in \mathbb{R}^N$  encodes the **uncertainty** level in each of the simplicial processes.
- ⇒ Number of unknown parameters is drastically **reduced** from  $N^2$  to  $N$ .
- ⇒ Enables **topology-aware** uncertainty modeling.

- with the **Dirac operator** defined as:

$$\mathbf{D}_{\mathcal{X}} = \begin{bmatrix} \mathbf{0} & \mathbf{B}_1 & \mathbf{0} \\ \mathbf{B}_1^\top & \mathbf{0} & \mathbf{B}_2 \\ \mathbf{0} & \mathbf{B}_2^\top & \mathbf{0} \end{bmatrix}$$

# Topology-aware state-space model

## ► State model

⇒ Discrete-time stochastic evolution governed by Hodge Laplacians:

$$\mathbf{x}_t = \tilde{\mathcal{L}} \mathbf{x}_{t-1} + \mathbf{q}_t, \quad \mathbf{q}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{D}_{\mathcal{X}} \text{diag}^2(\boldsymbol{\alpha}) \mathbf{D}_{\mathcal{X}})$$

⇒ where  $\tilde{\mathcal{L}} \simeq \mathbf{I} - c\delta t \mathcal{L}$  and uncertainty propagates via Dirac operator  $\mathbf{D}_{\mathcal{X}}$ .

## ► Observation model

⇒ We model partial, nonlinear measurements over a multi-hop neighborhood:

$$\mathbf{y}_t = \boldsymbol{\Phi} \mathbf{H}(\mathcal{L})(\mathbf{x}_t + \hat{f}(\mathbf{x}_t)) + \mathbf{n}_t$$

⇒  $\boldsymbol{\Phi} \in \{0, 1\}^{N_o \times N}$ : sampling matrix for partial observability

⇒  $\mathbf{H}(\mathcal{L})$ : topology-aware (Hodge/Dirac-induced) spectral filter

⇒  $\hat{f}(\cdot)$ : point-wise nonlinear mapping (parameterized via RFF)

# Inference and learning setup

## Discrete state-space model.

$$\mathbf{x}_t = \tilde{\mathcal{L}}\mathbf{x}_{t-1} + \mathbf{q}_t, \quad \mathbf{q}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{D}_{\mathcal{X}} \text{diag}^2(\boldsymbol{\alpha}) \mathbf{D}_{\mathcal{X}})$$

$$\mathbf{y}_t = \Phi \mathbf{H}(\mathcal{L})(\mathbf{x}_t + \hat{f}(\mathbf{x}_t)) + \mathbf{n}_t$$

$\Phi$ : masking,  $\mathbf{H}(\mathcal{L})$ : topological filter,  $\hat{f}(\cdot)$ : nonlinear mapping

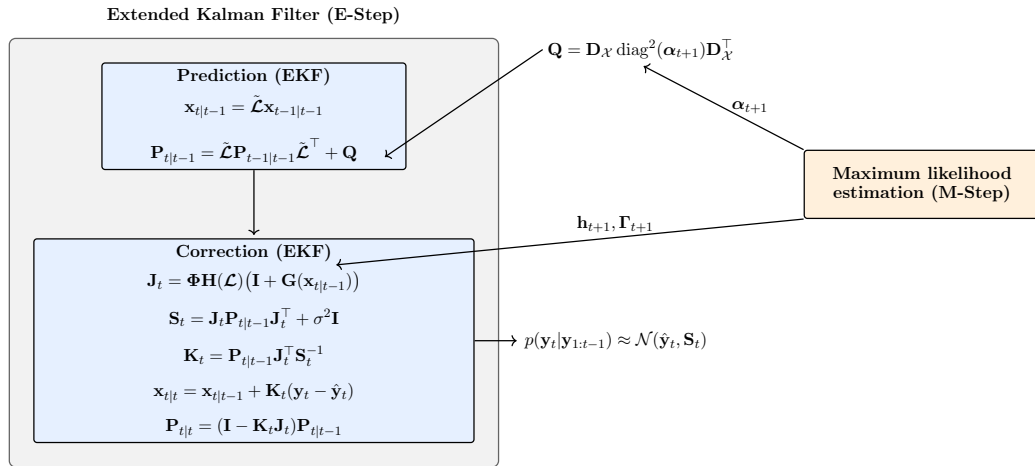
- **RFF Parameterization:** To retain expressivity while enabling online learning, we approximate  $f(\cdot)$  via Random Fourier Features (RFF):

$$\hat{f}(x_t[n]) \approx \boldsymbol{\Gamma}_n \mathbf{z}_v(x_t[n]), \quad \boldsymbol{\Gamma}_n \in \mathbb{R}^{1 \times 2M}$$

- **Online EM Framework:**

- ⇒ 1) **State estimation:** track  $\mathbf{x}_t$  via **EKF** (E-step).
- ⇒ 2) **Parameter learning:** update  $\boldsymbol{\alpha}, \mathbf{h}, \boldsymbol{\Gamma}$  via MLE (M-step).

# Joint parameter estimation and tracking



- Recursively perform state tracking (EKF) and parameter updates for  $\mathbf{h}$  (filter),  $\boldsymbol{\alpha}$  (uncertainty), and  $\boldsymbol{\Gamma}$  (nonlinearity).

# Online EM: E-step and M-step

- **E-step (EKF)**: Local linearization using the explicit RFF derivative.

$$\mathbf{J}_t = \Phi \mathbf{H}(\mathcal{L}) \left( \mathbf{I} + \mathbf{G}(\mathbf{x}_{t|t-1}) \right), \quad \text{where } \mathbf{G}_{nn} = \Gamma_n \frac{\partial \mathbf{z}_v(x_{t|t-1}[n])}{\partial x}$$

⇒ The innovation covariance is  $\mathbf{S}_t = \mathbf{J}_t \mathbf{P}_{t|t-1} \mathbf{J}_t^\top + \sigma^2 \mathbf{I}$ .

- **M-step (MLE)**: Instantaneous negative log-likelihood  $\ell_t(\boldsymbol{\alpha}, \mathbf{h}, \Gamma)$ :

$$\ell_t = \ell_{t-1} + \frac{1}{2} \log |\mathbf{S}_t| + \frac{1}{2} (\mathbf{y}_t - \hat{\mathbf{y}}_t)^\top \mathbf{S}_t^{-1} (\mathbf{y}_t - \hat{\mathbf{y}}_t)$$

- Online gradient updates (via Adam):

$$\boldsymbol{\alpha}_t = \boldsymbol{\alpha}_{t-1} - a_1 \nabla_{\boldsymbol{\alpha}} \ell_t$$

$$\mathbf{h}_t = \mathbf{h}_{t-1} - a_2 \nabla_{\mathbf{h}} \ell_t$$

$$\Gamma_t = \Gamma_{t-1} - a_3 \nabla_{\Gamma} \ell_t$$

# Online Cell Selection: Framework

- ▶ **Goal:** Dynamically infer relevant higher-order interactions.
- ▶ **Two-Mode Procedure:**
  - ⇒ **Identification:** Evaluate and activate candidate 2-cells via an accept/reject rule during EKF updates.
  - ⇒ **Tracking:** Fix the inferred topology for standard EKF recursion.
- ▶ **Binary Masking (Avoids combinatorial explosion):**
  - ⇒ Use a cell-activation mask  $\mathbf{e} \in \{0, 1\}^{N_2}$ :

$$\mathbf{B}_2 = \mathbf{B}_2^* \text{diag}(\mathbf{e})$$

- ⇒  $\mathbf{B}_2^*$  includes all candidates.
- ⇒ Deactivating a cell **zeros out** its column.
- ⇒ **Benefit:** Mutes topological influence while keeping matrix dimensions fixed.

# Uncertainty-Driven Scoring

- ▶ **Intuition:** Missing higher-order constraints cause the filter to assign **large uncertainty** to their boundary edges.
- ▶ **Process Covariance** (regularized by  $\gamma \mathbf{I}$  for observability):

$$\mathbf{Q} \simeq \delta t \mathbf{D} \text{diag}^2(\boldsymbol{\alpha}) \mathbf{D}^\top + \gamma \mathbf{I}$$

- ▶ **Scoring Metric:**

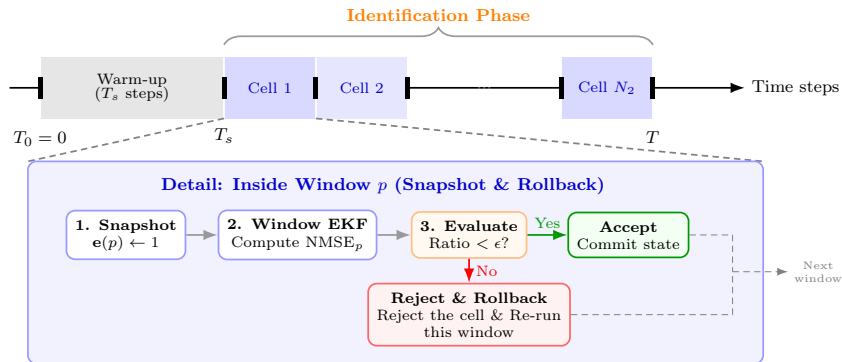
$\Rightarrow$  For a candidate 2-cell  $k$  (with boundary edges  $\mathcal{E}_k$ ), score by the average uncertainty of its edges ( $\boldsymbol{\alpha}^{(1)}$ ):

$$\xi_k = \frac{1}{|\mathcal{E}_k|} \sum_{j \in \mathcal{E}_k} (\alpha_j^{(1)})^2$$

$\Rightarrow$  **Larger**  $\xi_k \rightarrow$  High uncertainty  $\rightarrow$  High priority to activate cell  $k$ .

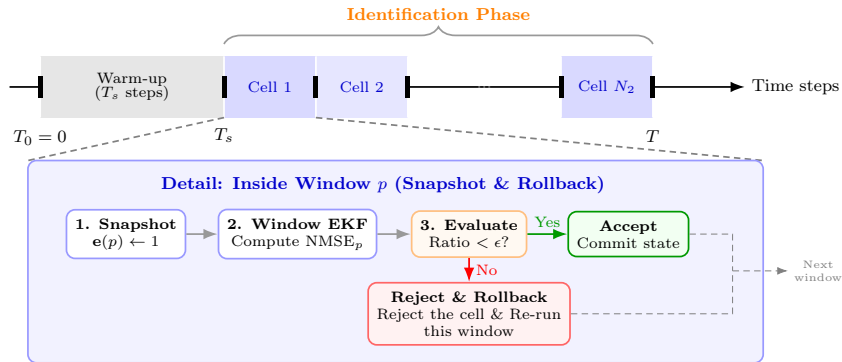
# Performance-Driven Selection with State Rollback

- **Challenge:** A wrong topological hypothesis ( $e_k = 1$ ) can affect the EKF state, leading to filter divergence.
- **Solution:** **Snapshot & Rollback** ensures state consistency during sequential testing.



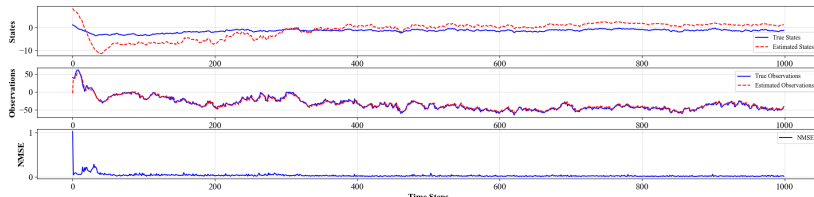


# Performance-Driven Selection with State Rollback



- ⇒ **Sequential Evaluation:** Post-warm-up, evaluate candidates across  $N_2$  discrete windows.
- ⇒ **State Caching & Trial:** Cache the EKF state and tentatively activate a candidate cell.
- ⇒ **Conditional Acceptance:** Accept if the forecasting  $NMSE_p$  reduction meets threshold  $\epsilon$ .
- ⇒ **Rollback & Re-run:** If rejected, **rollback** to the cached state and re-run EKF.

# Numerical Results: Forecasting node and edge signals



**Table:** NMSE of the forecasting. The best results are highlighted in **bold**.

| Method        | Synthetic          | Cherry             | WSN                | Capital Bikeshare  |
|---------------|--------------------|--------------------|--------------------|--------------------|
| TKF-N (Dirac) | <b>0.018±0.008</b> | 0.083±0.004        | <b>0.014±0.002</b> | 0.220±0.006        |
| TKF-L (Dirac) | 0.018±0.005        | 0.086±0.005        | 0.016±0.002        | 0.218±0.001        |
| TKF-N (Hodge) | 0.536±0.052        | 0.069±0.008        | 0.042±0.001        | 0.296±0.006        |
| TKF-L (Hodge) | 0.536±0.053        | 0.067±0.009        | 0.041±0.001        | 0.297±0.004        |
| TKF (empty)   | 0.080±0.039        | 0.086±0.004        | 0.014±0.002        | <b>0.210±0.004</b> |
| TOPO-LMS      | 0.065±0.033        | 0.031±0.001        | 0.017±0.010        | 0.270±0.001        |
| SC-VAR        | 0.535±0.067        | 0.149±0.079        | 0.039±0.004        | 0.307±0.008        |
| CC-VAR        | 0.044±0.002        | 0.085±0.002        | 0.122±0.010        | 0.705±0.006        |
| LSTM          | 0.121±0.005        | <b>0.030±0.002</b> | 0.022±0.005        | 0.222±0.006        |

## ► Take-Home Messages:

⇒ Cross-level method outperforms isolated processing.

⇒ TKF achieves the best results in WSN and Bicycle networks.

# Numerical Results: Forecast and Interpolate

**Table:** Performance (NMSE) of forecasting under partial observability across different datasets. The best results are highlighted in **bold**.

| Method        | 10%                | Synthetic<br>20%   | 30%                | 10%                | Cherry<br>20%      | 30%                |
|---------------|--------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
| TKF-N (Dirac) | <b>0.051±0.026</b> | <b>0.108±0.079</b> | <b>0.135±0.077</b> | 0.175±0.098        | 0.506±0.197        | 0.628±0.281        |
| TKF-L (Dirac) | 0.168±0.183        | 0.195±0.107        | 0.386±0.311        | 0.169±0.096        | 0.479±0.178        | 0.595±0.265        |
| TKF-N (Hodge) | 0.562±0.054        | 0.656±0.114        | 0.651±0.084        | <b>0.118±0.054</b> | <b>0.198±0.119</b> | 0.349±0.263        |
| TKF-L (Hodge) | 0.569±0.058        | 0.653±0.114        | 0.624±0.089        | 0.136±0.059        | 0.207±0.105        | <b>0.309±0.182</b> |
| TKF (empty)   | 0.247±0.088        | 1.028±1.196        | 1.174±0.994        | 0.178±0.100        | 0.499±0.199        | 0.638±0.296        |

| Method        | 10%                | WSN<br>20%         | 30%                | 10%                | Capital Bikeshare<br>20% | 30%                |
|---------------|--------------------|--------------------|--------------------|--------------------|--------------------------|--------------------|
| TKF-N (Dirac) | 0.120±0.038        | 0.218±0.079        | 0.360±0.091        | 0.903±0.323        | 1.167±0.325              | 1.489±0.242        |
| TKF-L (Dirac) | 0.115±0.021        | <b>0.194±0.085</b> | <b>0.250±0.062</b> | 0.999±0.588        | 1.120±0.378              | 1.399±0.293        |
| TKF-N (Hodge) | <b>0.099±0.037</b> | 1.153±2.082        | 4.835±7.548        | 0.329±0.037        | 0.796±0.558              | 0.659±0.328        |
| TKF-L (Hodge) | 0.164±0.080        | 0.258±0.120        | 3.747±5.399        | <b>0.318±0.012</b> | <b>0.707±0.456</b>       | <b>0.584±0.396</b> |
| TKF (empty)   | 0.103±0.035        | 0.201±0.068        | 0.321±0.083        | 0.688±0.176        | 0.782±0.163              | 1.290±0.399        |

► **Take-Home:** TKF-L prevents error explosion under severe missing data.

# Numerical Results: Cell Identification

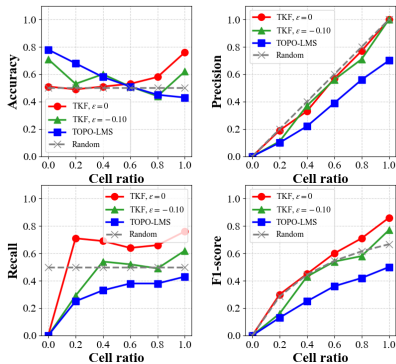


Table: Performance (NMSE) of forecasting for the inferred structure.

| Dataset           |      | Observability     |                   |                   |
|-------------------|------|-------------------|-------------------|-------------------|
|                   |      | Empty             | Inferred          | Full              |
| Synthetic         | 0%   | 0.044 $\pm$ 0.020 | 0.042 $\pm$ 0.020 | 0.041 $\pm$ 0.020 |
|                   | 20%  | 0.052 $\pm$ 0.025 | 0.038 $\pm$ 0.016 | 0.035 $\pm$ 0.017 |
|                   | 40%  | 0.076 $\pm$ 0.038 | 0.036 $\pm$ 0.014 | 0.031 $\pm$ 0.012 |
|                   | 60%  | 0.081 $\pm$ 0.033 | 0.034 $\pm$ 0.015 | 0.027 $\pm$ 0.012 |
|                   | 80%  | 0.083 $\pm$ 0.047 | 0.034 $\pm$ 0.016 | 0.023 $\pm$ 0.007 |
|                   | 100% | 0.080 $\pm$ 0.039 | 0.026 $\pm$ 0.013 | 0.018 $\pm$ 0.008 |
| Cherry            |      | 0.086 $\pm$ 0.004 | 0.085 $\pm$ 0.004 | 0.083 $\pm$ 0.004 |
| WSN               |      | 0.014 $\pm$ 0.002 | 0.014 $\pm$ 0.002 | 0.014 $\pm$ 0.002 |
| Capital Bikeshare |      | 0.210 $\pm$ 0.004 | 0.216 $\pm$ 0.004 | 0.220 $\pm$ 0.006 |

## ► Key Findings:

⇒ **Superior Inference:** TKF peaks at an **F1-score of 0.86**.

⇒ **Information Density:** The F1-score increases monotonically as the cell ratio grows.

# Learning Activity: Why Topology in State-Space Models?

State-space models track how a hidden state evolves over time. For topological data, the state may live on nodes, edges, faces, or cells, and its evolution should respect the incidence structure and the coupled interactions across dimensions.

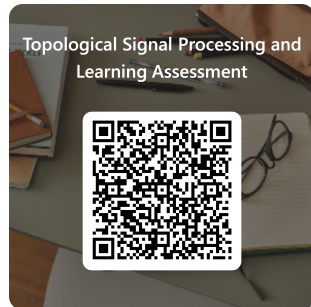
- ▶ **Q8.1:** You are modeling a dynamic flow signal on the edges of a road network. The flow evolves over time and is affected both by node-level divergence at intersections and by local circulations around triangular regions. Which modeling choice best captures this structure?

- ▶ **Instructions**

- ⇒ Scan the QR code to open the activity
- ⇒ Work individually first
- ⇒ Then discuss your answer in pairs or groups of 3
- ⇒ Submit your final answer in Microsoft Forms

- ▶ **Logistics**

- ⇒ **Time:** 10 min
- ⇒ **Goal:** Understand why topological state-space models need operators beyond graph adjacency
- ⇒ **Product:** One submitted response per student + justification to peers



<https://forms.office.com/>

# Further Reading

- ▶ **Graph state space model:**

- ⇒ Sabbaqi, M., Taormina, R., Isufi, E. (2025). GKNet: Graph Kalman Filtering and Model Inference via Model-based Deep Learning. arXiv preprint arXiv:2506.22004.

- ▶ **Topological state space model:**

- ⇒ Money, R., Sabbaqi, M., Krishnan, J., Beferull-Lozano, B., Isufi, E. (2024, October). Kalman filtering for simplicial processes. In 2024 58th Asilomar Conference on Signals, Systems, and Computers (pp. 49-53). IEEE.